Music Piece Classification

Selina Zou, Phillip Duarte, Ethan Fan

STAT 4830

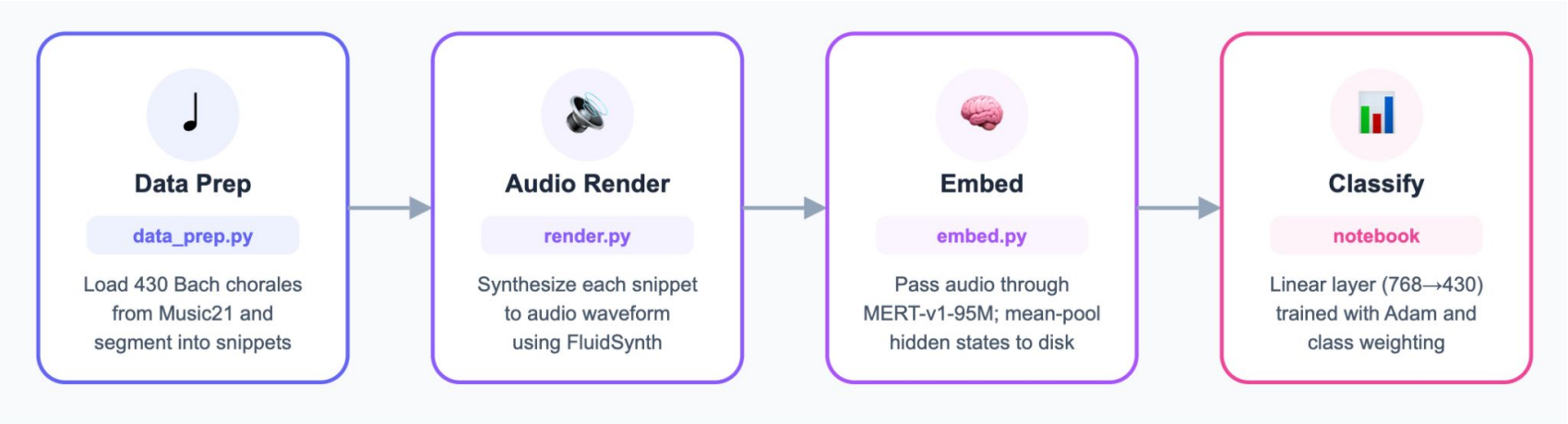
Friday, March 20, 2026

Data Perturbation

- Goal: Introduce pitch shifting, time stretching to the FluidSynth renders to prevent the model from "memorizing" clean audio.
  - Also to allow more data per class (i.e. piece)
- Use librosa library's 'effects' on the WAV files that are the audio waveforms
- To also apply on MIDI of jazz/symphonies

Data Perturbation: Pipeline Additions

Previously:



Data Perturbation: Pipeline Additions

Now:
Can enter pipeline with music21/MIDI/WAV
(see perturb.py which calls some methods in prev files)

1. Load MIDI: direct MIDI or music21 (do conversion)
2. Render WAV: from the MIDIs
3. Perturb WAVs: using librosa
4. Segment into snippets with train/test split → manifest.csv
5. Embed snippets similarly

Data Perturbation: Methods

Tempo Change

- time_stretch(y, rate): y is the np.ndarray waveform and rate is float (0.9 = 90% tempo)
- [0.9, 0.95, 1.0, 1.05, 1.1]

Pitch Shift

- pitch_shift(y, sr, n_steps): same y, sr is sample rate (samples/s), n_steps is semitones up/down
- [0, -1, 1, -2, 2]

Combinations yield 25 versions, total 430 * 25 = 10750 WAVs.

Data Perturbation: Snippets

- Previously:
  - Split by measures (4 measures, overlap option)
  - 2992 snippets / 430 pieces = <u>7 snippets/piece</u>
- Now
  - Split by duration (8 sec, 4 sec overlap)
  - Rationale: for Bach chorales, snippets ~ 8 sec
  - 10750 "pieces" * 7 snippets = 75250 snippets
    → <u>175 snippets/piece</u>

Data Perturbation: Example (BWV 1)

bach__bwv1.6__t1.0p0.wav
- Original

bach__bwv1.6__t1.1p2.wav
- Tempo = 10% faster
- Pitch = 2 semitones up

bach__bwv1.6__t0.9p-2.wav
- Tempo = 10% slower
- Pitch = 2 semitones down

Larger Dataset

- 220 (MIDI file) movements from classical symphonies from Classical Music Archive
- Combined with perturbations -> magnitudes larger dataset
- Still need to label pieces for composer-level classification

Deployment on PrimeIntellect: Model

- Our goal is to leverage PrimeIntellect's compute to perform computationally extensive gridsearch over MLP configurations
- To accomplish this, we do the following:
  - Take the notebook and turn it into a runnable python script with uv for package management
  - Use PrimeIntellect's On-Demand Cloud pod instances to run the script
  - Have cleaned and ready data (post data_prep.py and render.py from our pipeline) stored on persistent storage for quick access in different instances

Deployment on PrimeIntellect: Data

- Data pipeline takes a somewhat non-trivial amount of time to process our data and then turn it into snippets (1.5 hrs for aforementioned perturb., lots more for snippeting)
- As such, as a part of our effort to utilize PrimeIntellect, we will also be moving our data pipeline to prime intellect using its cheap CPU nodes to write the processed data directly into our persistent storage.
- This frees up our computers and CPU node is so cheap we that we don't need to monitor it super closely either.

Continued Work

- **Address the "Data Sparsity" Problem**
  - Expand Dataset: Move beyond the 430 Bach chorales to a fuller IMSLP corpus or the Million Song Dataset.
  - Use perturbed data in training, create more kinds of perturbations
- **Architectural Search**
  - Identify Current Best: Gridsearch over MLP configurations to find most accurate hyperparameters
  - Explore Other Models: Replace mean-pooling with a Temporal Transformer Encoder or Bi-LSTM.
- **Deep Fine-Tuning**
  - Unfreeze MERT: Transition from "Feature Extraction" to "Fine-Tuning."
  - Metric Learning: Use a Triplet Loss or Contrastive Learning (CLAP) approach. Instead of classifying 430 labels, train the model to push embeddings of the same piece together and pull different pieces apart.